Link to Github Repository:
https://github.com/jdaltongit/CSCI580_Jesse_Dalton

Problem Context:

The Traveling Salesman Problem (TSP) is an optimization problem that tries to find the shortest possible route that visits every city exactly once and returns to the starting point. It is difficult to solve exactly because the number of possible routes grows extremely large as the number of cities increases, making it computationally infeasible to check every option. Instead we use heuristic methods like Simulated Annealing and Genetic Algorithms, which cut down search time by balancing when to explore new solutions and when to exploit good ones already found.

Solutions: Simulated Annealing

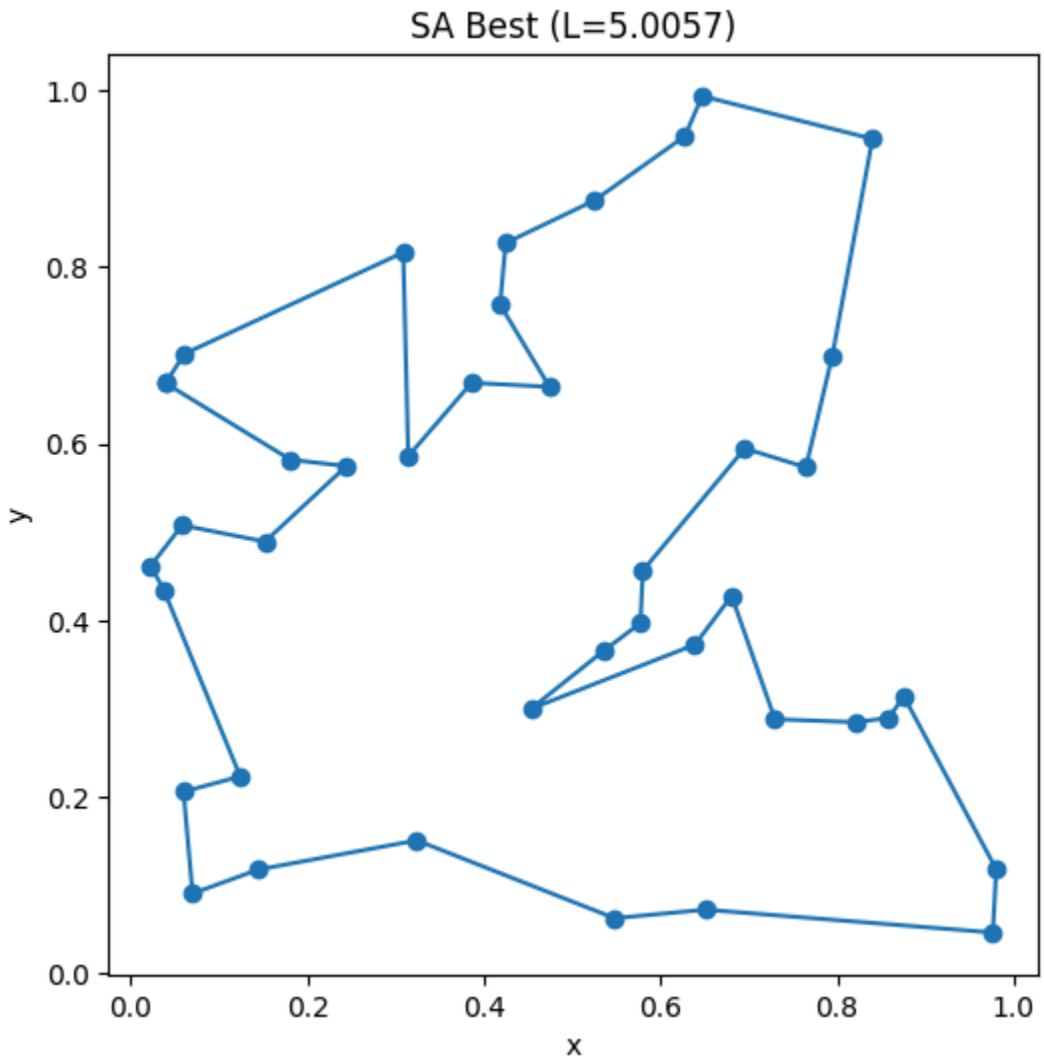
The SA implementation starts with the nearest neighbor tour as its initial solution. Each iteration it generates a random 2-opt neighbor by reversing a segment of the current tour. It then decides whether to accept the new tour based on the acceptance rule: if the new tour is shorter, it is always accepted, if it is longer it is accepted with probability exp(-delta/T) where delta is how much worse it is and T is the current temperature. The key code for this is:

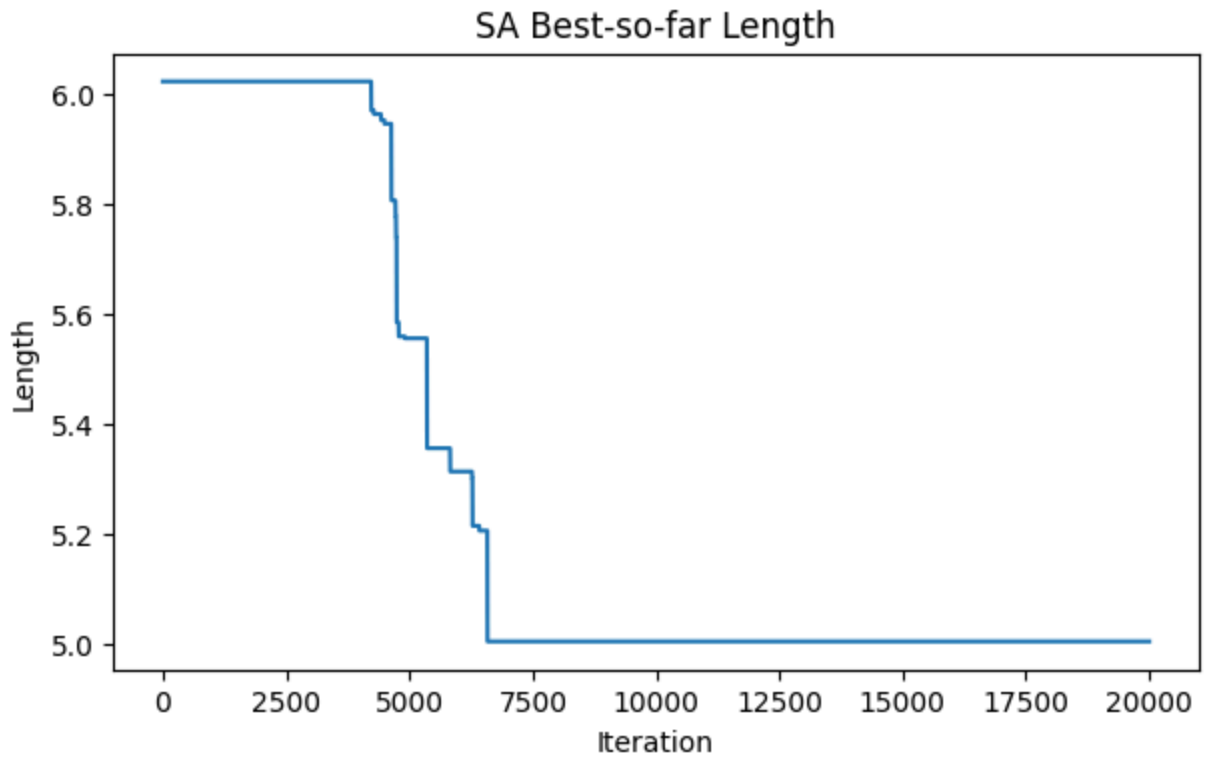
```
if delta <= 0:

    accept = True

else:

    accept = rng.random() < math.exp(-delta / T)
```

After each iteration the temperature is reduced using geometric cooling, multiplying T by alpha each step and clamping it so it never reaches zero:

```
T = T * cfg.alpha

T = max(T, 1e-12)
```

The parameters used were: iters=20000, t0=0.2, alpha=0.9995, achieving a best tour length of 5.0057, a 16.87% improvement over the nearest neighbor baseline of 6.0214.





Solutions: Genetic Algorithm

The GA implementation starts with a population of 300 tours, seeded with the nearest neighbor tour and filled with random tours. Each generation it selects parents using tournament selection, where 3 random individuals compete and the shortest tour wins:

```
candidates = rng.sample(range(len(pop)), k)

winner = min(candidates, key=lambda i: lengths[i])

return pop[winner][:]
```

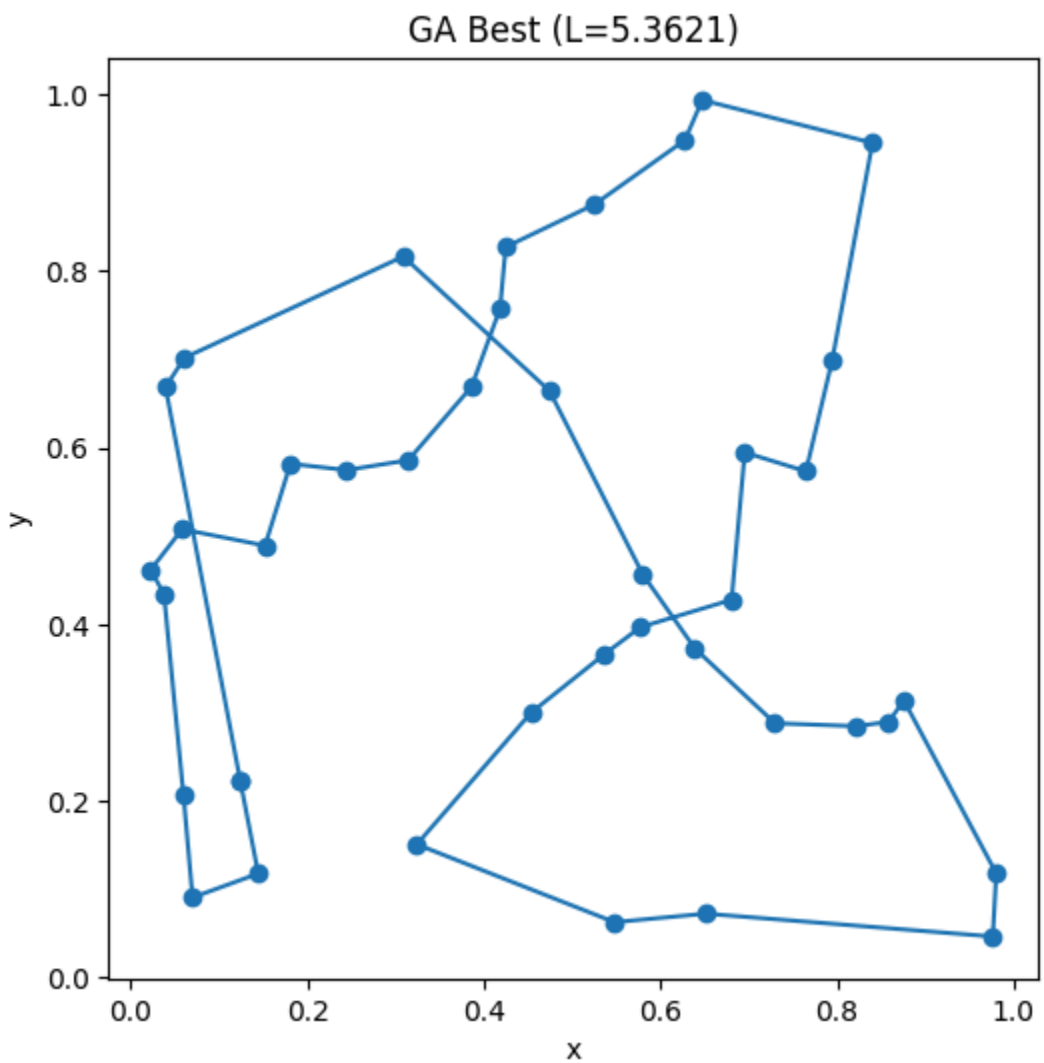
Parents are combined using order crossover (OX), which copies a random slice from parent1 and fills the remaining positions using parent2's order while skipping duplicates:

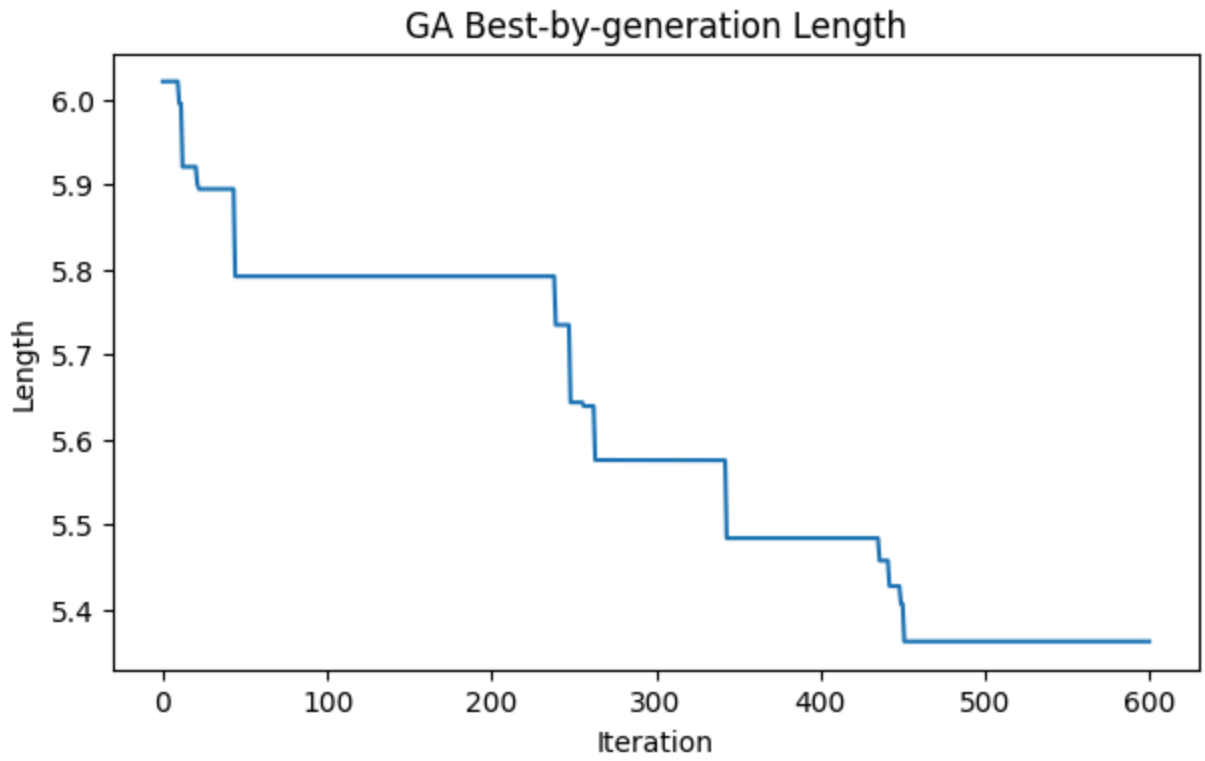
```
child = [None] * n

child[i:j+1] = parent1[i:j+1]
```

placed = set(parent1[i:j+1])

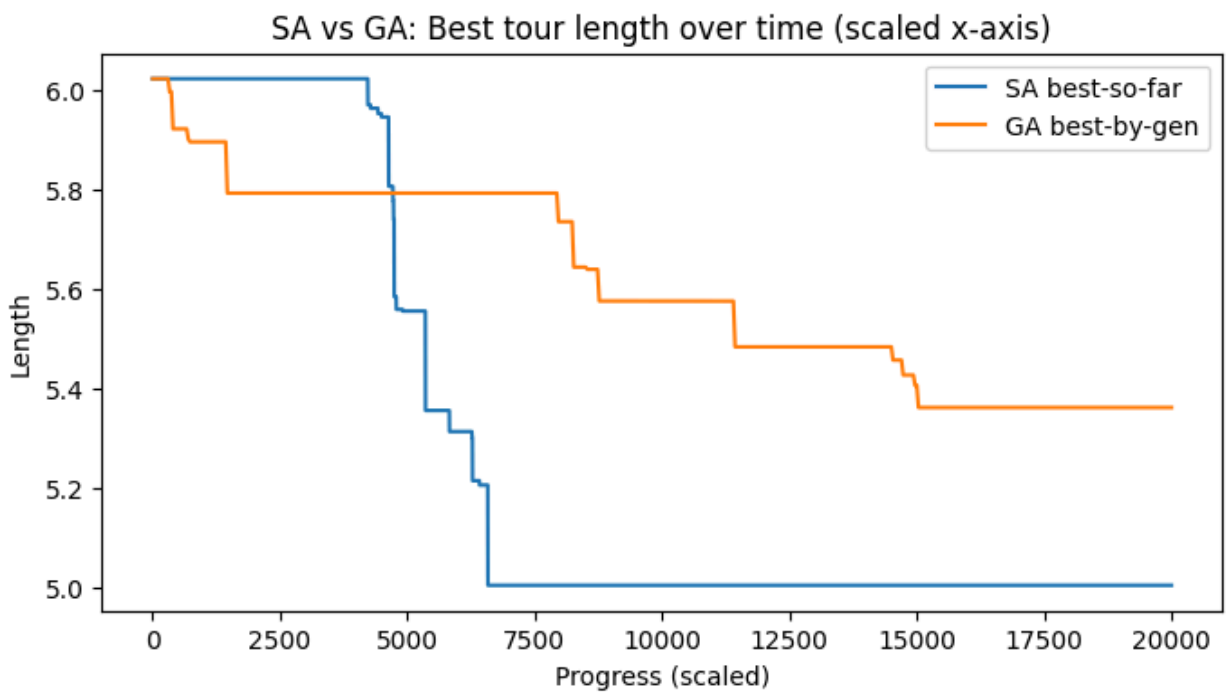
p2_order = [city for city in parent2 if city not in placed]

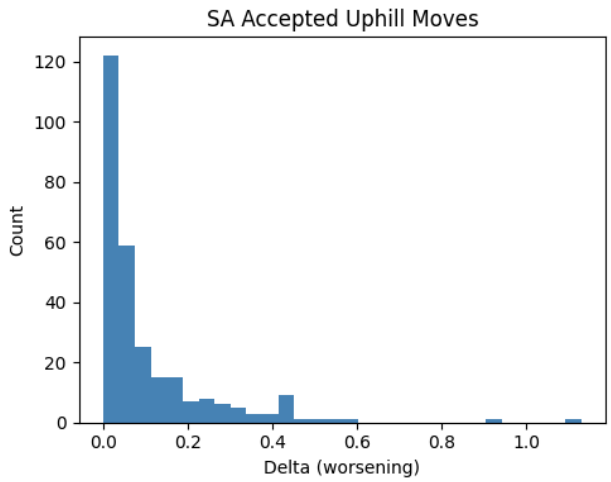
Swap mutation is then applied with a 50% chance, randomly swapping two cities in the tour to maintain diversity: `mutated[i], mutated[j] = mutated[j], mutated[i]` The top 5 tours are carried over unchanged each generation. The parameters used were: pop_size=300, generations=600, tournament_k=3, mutation_rate=0.5, achieving a best tour length of 5.3621, a 10.95% improvement over the nearest neighbor baseline.

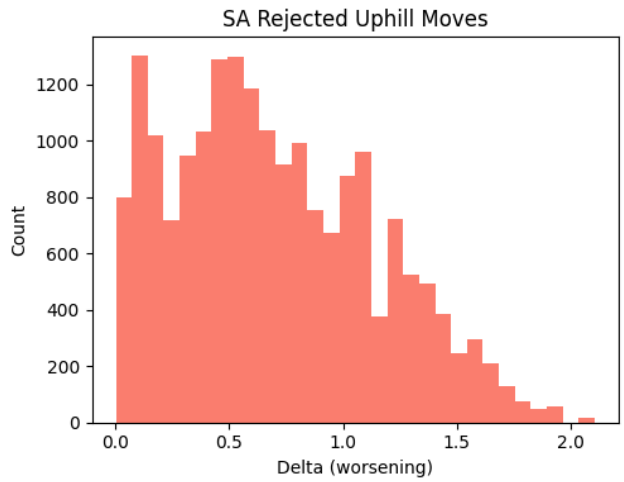


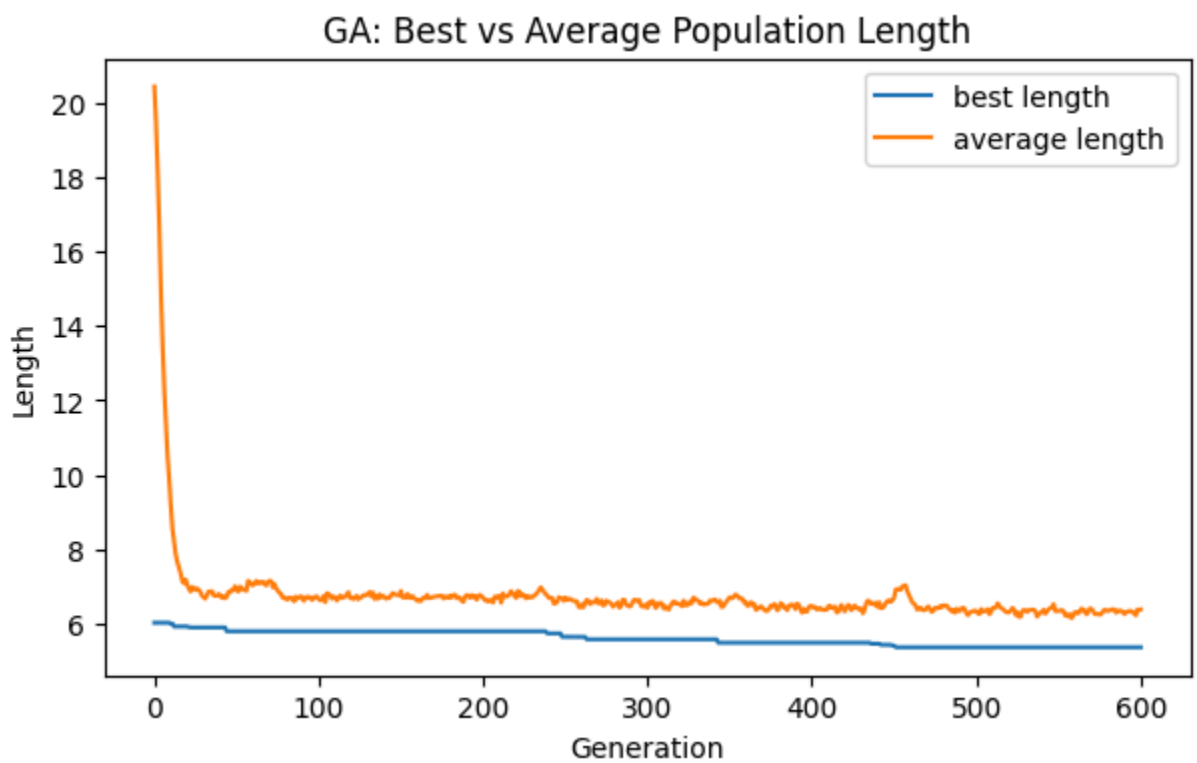


Experimental Results & Analysis









Algorithm Comparison:
NN: 6.0214 | SA: 5.0057 | GA: 5.3621
SA improvement: 16.87%
GA improvement: 10.95%

SA accepted uphill moves: 283
SA rejected uphill moves: 19392

Grade Check:
SA length: 5.0057 | Improvement vs NN: 16.87%
✅ SA passed improvement threshold!
GA length: 5.3621 | Improvement vs NN: 10.95%
✅ GA passed improvement threshold!

- **Q1:** Which algorithm converged faster?
- **A1:** SA converged to a better solution faster. Looking at the plot_compare chart, SA had a sharp drop around progress 5000-7500 and reached its best length of 5.0057. GA was still gradually improving at the end of 600 generations at 5.3621.
- **Q2:** Which showed more variability?
- **A2:** GA showed more variability. Its curve had multiple small steps down across all 600 generations. SA dropped sharply once around progress 5000-7500 then flatlined for the rest of its iterations.
- **Q3:** Did SA escape local minima? Evidence?
- **A3:** Yes. The accepted uphill moves histogram shows SA accepted 283 worsening moves. These were mostly small delta values, meaning early on when temperature was high SA occasionally accepted worse solutions which allowed it to escape local minima and find a better tour than GA.
- **Q4:** Did GA lose diversity? Evidence?
- **A4:** Yes. The best vs average population plot shows the average collapsed quickly in the first 25 generations but stayed above the best for all 600 generations. This means the population lost diversity early but never fully converged to a single solution.

Overall, SA outperformed GA on this TSP instance, achieving a 16.87% improvement over the nearest neighbor baseline compared to GA's 10.95%. SA was more effective because its temperature based acceptance rule allowed it to escape local minima early on, while GA struggled with diversity loss despite parameter tuning to meet the minimum improvement threshold.